Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: S-04

Team No.: 15

Project Title: System Calls

Team Members

Roll Number	Student Name	Signature
2520030259	Ch. Yashwanth	
2520030259	V. Bharath	
2520030186	B. Praneeth	

1. Abstract

The project is a **Multi-User Chat Application developed in C on Linux/Ubuntu using TCP client-server architecture**. A central server accepts multiple clients and manages communication between connected users. The application demonstrates how multiple Linux system-programming concepts can work together in a practical network application.The main academic objective is to demonstrate **Linux system calls, process management, inter-process communication, networking, signal handling, and resource management**. The project uses `fork()` to handle multiple clients, `pipe()` for communication between processes, and TCP socket system calls for network communication.The project is designed primarily as an **Operating Systems and System Programming learning application**, providing practical understanding of how Linux system calls are used to build a concurrent multi-user network application.

2. Problem Statement

Existing networked chat applications rely on high-level frameworks and services that abstract the underlying operating-system mechanisms involved in process management, inter-process communication, and network communication. This creates a gap in practically understanding how Linux system calls coordinate these mechanisms in a concurrent multi-user application. Therefore, the project aims to develop a C-based TCP chat application on Linux/Ubuntu that directly utilizes Linux system calls to demonstrate process creation, IPC, socket communication, signal handling, and resource management in an integrated system.

3. Objectives

- To understand Linux system calls and their role in Operating Systems.
- To implement a multi-user TCP chat application using C.
- To demonstrate process creation using `fork()`.
- To demonstrate inter-process communication using `pipe()`.
- To implement TCP communication using Linux socket APIs.
- To demonstrate concurrent handling of multiple clients.
- To understand signal handling using Linux APIs.
- To demonstrate proper process and resource management.

4. Proposed methodology

The proposed system is implemented in **C on Linux/Ubuntu** using a TCP client-server architecture.

1. Create a TCP server using socket().
2. Configure the server using bind().
3. Put the server into listening mode using listen().
4. Accept client connections using accept().
5. Create a child process using fork() for each connected client.
6. Use pipe() for inter-process communication between child and parent processes.
7. Use send() and recv() for exchanging messages.
8. Implement message routing for:
    - Broadcast messages
    - Private messages
9. Use waitpid() for child-process management.
10. Use sigaction() and kill() for signal handling.
11. Use close() to properly release sockets and resources.

User Features

- Username/login
- Multiple simultaneous users
- /users
- /broadcast <message>

- /msg <user> <message>
- /help
- /quit

Server Workflow

socket() → bind() → listen() → accept() → fork() → communicate → cleanup

Client Workflow

socket() → connect() → send()/recv() → close()

5. Operating Systems Concepts / Linux APIs Used

System Call / API	Purpose
socket()	Creates a TCP communication endpoint
bind()	Assigns an address and port to the server socket
listen()	Enables the server to listen for client connections
accept()	Accepts incoming client connections
connect()	Connects a client to the server
fork()	Creates a child process to handle a client
pipe()	Enables inter-process communication
send()	Sends messages through a TCP socket
recv()	Receives messages through a TCP socket
waitpid()	Manages child processes
sigaction()	Configures signal handling
kill()	Sends signals to processes
close()	Closes sockets and releases resources

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030259	Yashwanth	Handles IPC, message routing, testing, and documentation.
2520030169	Bharath	Handles the client-side communication and user commands.
2520030186	Praneeth	Handles the server and process management

7.Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform	Purpose
Linux / Ubuntu	Operating environment
Windows + WSL2	Running Ubuntu on Windows if required
C	Programming language
GCC	C compiler
POSIX/Linux System Calls	OS-level programming
BSD/POSIX Sockets	TCP networking
Linux/Ubuntu Terminal	Compilation and execution
Make / Makefile	Build automation
Git / GitHub	Version control and project repository
Wi-Fi / LAN	Multi-laptop network demonstration

8. Expected Outcome

The project is expected to deliver a functional multi-user TCP chat application that demonstrates Linux system calls, concurrent client handling, inter-process communication, signal handling, and proper resource management while providing practical understanding of Operating Systems and system programming concepts**.**

9. GitHub Repository Link: https://github.com/yash-290807/KLH-CSE-2026-2027-Team15-System-Calls.git

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature
Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____